Networks Assignment - 04 Lab Report
Group 07

Packet Scheduling Algorithms — WFQ and FCFS

In modern computer networks, multiple data flows often share a common output link or transmission channel. To manage this contention and ensure efficient bandwidth utilization, **packet scheduling algorithms** are employed at routers or switches. These algorithms determine the order in which packets are transmitted from the buffer to the output link, directly affecting **delay**, **fairness**, and **throughput**.

This experiment focuses on implementing and comparing two fundamental scheduling strategies - **First-Come-First-Serve (FCFS)** and **Weighted Fair Queuing (WFQ)** through simulation.

1. Objective

The objective of this experiment is to:

1. Implement and simulate two packet scheduling algorithms - **FCFS** and **WFQ**.
2. Analyze their performance under identical network conditions.
3. Measure system-level metrics such as:
    - **Server Utilization**
    - **Average Packet Delay**
    - **Packet Drop Probability**
    - **Fairness Index (Jain's Index)**
4. Understand the trade-offs between simplicity (FCFS) and fairness (WFQ).

2. Background Theory

2.1 Packet Scheduling in Computer Networks

Routers and switches often receive packets faster than they can transmit them through an outgoing link. When this happens, packets are queued in a buffer. The **packet scheduler** decides which packet to send next when the link becomes free.
The design of a good scheduler is crucial for:

- Minimizing delay and packet loss.
- Ensuring fair bandwidth allocation among competing flows. ● Maintaining high link utilization.

Different scheduling policies balance these factors differently, making them suitable for specific network conditions and application requirements.

2.2 First-Come-First-Serve (FCFS)

First-Come-First-Serve (FCFS), also known as **FIFO (First-In-First-Out)**, is the simplest scheduling policy.

Working Principle:

- Packets are transmitted in the exact order they arrive in the queue.
- No distinction is made between packets from different sources.
- If the buffer is full, newly arriving packets are dropped.

Characteristics:

- Simple to implement.
- Ensures basic order preservation (no starvation).
- Does **not** guarantee fairness when flows have unequal rates or packet sizes.
- Can lead to **head-of-line blocking**, where large packets delay smaller ones.

Advantages:

- Minimal computational overhead.
- Suitable for low-load networks or single-flow systems.

Disadvantages:

- Unfair bandwidth allocation under heavy traffic.
- Poor performance for latency-sensitive flows when mixed with large bulk transfers.

2.3 Weighted Fair Queuing (WFQ)

Weighted Fair Queuing (WFQ) is a sophisticated scheduling algorithm designed to ensure fair bandwidth distribution among multiple flows, based on assigned weights.

Concept:

WFQ is an approximation of Generalized Processor Sharing (GPS), a theoretical model that divides link capacity continuously among active flows proportional to their weights.

Each flow *i* is assigned a weight w_i, determining the share of link capacity it should receive.

Core Mechanism:

1. Each packet is assigned a **finish time (F_i)** when it enters the queue:

$$F_i = \max(F_{prev}, A_i) + \frac{L_i}{w_i}$$

where
A_i = arrival time,
L_i = packet length,
w_i = weight (priority).

2. The scheduler always selects the packet with the **smallest finish time** for transmission.
3. If the buffer is full, packets with the **largest finish number** (lowest priority) may be dropped.

Advantages:

- Provides fair bandwidth sharing among active flows.

- Prevents starvation of low-rate or low-priority sources. ● Supports QoS (Quality of Service) differentiation.

Disadvantages:

- Computationally more complex than FCFS.
- Requires precise timestamping and sorting operations.

2.4 Optimized Weighted Fair Queuing (Demers–Keshav–Shenker Algorithm)

The version of WFQ implemented in this lab is based on the classical algorithm proposed by **A. Demers, S. Keshav, and S. Shenker in their seminal paper *"Analysis and Simulation of a Fair Queueing Algorithm"* (ACM SIGCOMM, 1989).** This algorithm refines the idealized Generalized Processor Sharing (GPS) model into a practical, packet-based scheduling approach suitable for discrete-time systems.

Background

While GPS provides a perfectly fair service by continuously sharing the output link among flows, it is not implementable in real systems since packets cannot be divided. The Demers–Keshav–Shenker (DKS) WFQ algorithm approximates GPS behavior by assigning each packet a finish time, determining the order of transmission.

Each packet's finish time reflects when it would complete transmission under the ideal GPS model. The scheduler always transmits packets in the order of increasing finish times, ensuring near-perfect fairness while maintaining discrete packet boundaries.

Algorithmic Optimization

The DKS WFQ algorithm optimizes fairness computation and scheduling as follows:

1. Virtual Time Concept

    - A *virtual clock* V(t) is maintained, representing the normalized progress of the system.
    - V(t) increases only when packets are being transmitted.
    - It helps synchronize all flows relative to their weights and arrival times.

2. Finish Number Calculation For a packet k of flow i:

$$F_{i,k} = \max(V(a_{i,k}), F_{i,k-1}) + \frac{L_{i,k}}{w_i}$$

where:

`a_{i,k}` = packet arrival time
`L_{i,k}` = packet length
`w_i` = flow weight
`F_{i,k-1}` = finish time of previous packet in the same flow
`V(a_{i,k})` = system virtual time when the packet arrives

3. This ensures that a new packet's finish time depends both on its arrival time and its flow's previous load.
4. Scheduling Rule:
    - The scheduler always transmits the packet with the smallest finish time (F).
        - This ensures proportional fairness according to flow weights.
5. Buffer Management
    - If the output buffer reaches full capacity (B packets), the system drops the packet with the largest finish time (i.e., the least urgent one).
    - This prevents unfair dropping of low-weight or slow flows and improves fairness under congestion.

Implementation Details in the Code

The simulation models this optimized WFQ algorithm using the following steps:

- Multithreaded Traffic Generation:

Each source runs on a separate thread, generating packets at its configured rate P_i with random sizes in the range [L_min_i, L_max_i].

Packets are timestamped and assigned their finish numbers immediately upon creation.

- Central Scheduler Thread:

  A shared scheduling thread maintains a priority queue (min-heap) sorted by finish times.

  - The scheduler dequeues the packet with the smallest F_i .
  - Transmission delay is computed as L_i / C, where C is link capacity.
  - The virtual time V(t) is updated after each transmission.

- Fairness-Optimized Dropping Policy:

  When the buffer is full, the packet with the largest finish number (max(F)) is dropped.

  This aligns with the DKS optimization that preserves fairness among active flows.

- Performance Metrics Computation:

  The code continuously records:

  - Transmission timestamps (for delay calculation)
  - Per-flow throughput (for Jain's Fairness Index)
  - Dropped packet counts (for drop probability)
  - Link utilization statistics (for server utilization)

Advantages of the Optimized WFQ Implementation

- Achieves GPS-level fairness with discrete packets.
- Maintains bounded delay and low drop probability for low-rate or high-priority flows.
- Adaptable to varying load levels with minimal starvation.
- Computationally efficient using priority queues (O(log N) operations).

Limitations

- Requires precise virtual time management and synchronization across threads.
- Slight approximation errors may occur under very high burstiness due to thread scheduling and buffer contention.

Key References

A. Demers, S. Keshav, and S. Shenker,
"Analysis and Simulation of a Fair Queueing Algorithm," ACM SIGCOMM, 1989.

2.5 Performance Metrics

To evaluate and compare FCFS and WFQ, the following performance parameters are measured:

Metric	Description / Formula
Server Utilization	Fraction of total simulation time during which the link is busy transmitting packets.
Average Packet Delay	Mean time difference between packet generation and transmission completion.
Packet Drop Probability	$P_{drop} = \frac{Packets_{dropped}}{Packets_{generated}}$
Fairness Index (Jain's)	$F = \frac{(\sum x_i)^2}{n \times \sum x_i^2}$ where x_i is throughput of source *i*. Measures fairness of bandwidth allocation.

2.6 Comparison Summary

Feature	FCFS	WFQ
Scheduling Basis	Arrival time	Weighted fair sharing
Fairness	Low	High
Complexity	Low	Moderate to High
Delay Behavior	May vary with flow size	More predictable
QoS Support	None	Supported via weights
Best suited for	Simple or homogeneous traffic	Mixed or prioritized traffic

2.7 Theoretical Expectations

- **WFQ** is expected to achieve **higher fairness** and **lower drop probability**, especially when sources have unequal packet rates or sizes.
- **FCFS** may show **lower delay** under light traffic but performs poorly when load increases.
- **WFQ** ensures more balanced throughput distribution at the cost of slightly higher scheduling overhead.

2.8 Applications

- **FCFS**: Simple router queues, unmanaged switches, or legacy systems.
- **WFQ**: Quality-of-Service (QoS) networks, VoIP traffic management, multimedia streaming, and differentiated service architectures.

2.9 Summary

- Packet scheduling plays a central role in determining overall network performance.
- While **FCFS** emphasizes simplicity, **WFQ** provides controlled fairness and predictable service differentiation.
- This lab demonstrates how different scheduling policies influence performance metrics such as delay, throughput, and fairness under shared bandwidth conditions.

3. Implementation

3.1 Overview

The simulation was implemented in **C++17** using a **multithreaded architecture**. Each traffic source runs in its own thread to emulate concurrent packet generation, while a central scheduler handles packet transmission using either the **First-Come-First-Serve (FCFS)** or **Weighted Fair Queuing (WFQ)** algorithm.

The simulation computes system-level performance metrics such as:

- **Server Utilization**
- **Average Packet Delay**
- **Packet Drop Probability**
- **Jain's Fairness Index**

All parameters are configurable through an input file, as defined in the lab specification.

3.2 Simulation Architecture

Components

1. **Traffic Sources (Producers):**
    - Each source generates packets according to its rate Pi , packet size range [Lmin,i,Lmax,i] and active time interval [tbi,tei].
    - Packet arrivals follow an **exponential inter-arrival distribution**, ensuring realistic Poisson-like traffic behavior.

- Implemented using C++ threads - each source runs concurrently and populates its packet list.

2. **Packet Structure:**
   Each packet is represented by:

```
struct Packet {
    uint64_t id;
    int source;
    int size;
    double arrival;
    double start_tag;
    double finish_tag;
};
```

start_tag and **finish_tag** are used exclusively in WFQ scheduling.

3. **Scheduler (Central Consumer):**

- Maintains a shared buffer (queue) of limited capacity (**B**).
- Reads the merged, sorted list of all generated packets (sorted by arrival time).
- Processes packets based on the selected scheduling discipline (FCFS or WFQ).

4. Metrics Module:

- Tracks key simulation statistics such as busy time, total delay, drop count, and per-source throughput.
- At the end of the simulation, it calculates:

- **Server Utilization:** $U = \frac{\text{Total Busy Time}}{T}$
- **Average Packet Delay:** $D = \frac{\sum_i (t_{finish,i} - t_{arrival,i})}{N_{transmitted}}$
- **Drop Probability:** $P_{drop} = \frac{N_{dropped}}{N_{generated}}$
- **Fairness Index (Jain's):**

$$F = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}$$

where x_i = bytes transmitted by flow i.

3.3 FCFS Implementation

The **FCFS** scheduler uses a simple queue (**deque**) where packets are transmitted in arrival order.

Key Steps:

1. On arrival:
    - If the buffer is not full, enqueue the packet.
    - If full, drop the new packet (tail-drop policy).

2. The scheduler transmits packets one at a time in FIFO order.
3. Transmission delay is computed as:

$$t_{service} = \frac{\text{packet size}}{C}$$

Time Complexity:

- O(1) per enqueue/dequeue.

Advantages:

- Extremely simple; ideal for low-load conditions.

Limitations:

- No fairness; large packets block small ones (head-of-line blocking).

3.4

WFQ Implementation (Optimized DKS Version)

The **WFQ** scheduler is implemented as per the **Demers–Keshav–Shenker (1989)** algorithm.

Key Mechanisms:

1. **Virtual Time Tracking:**
   - Maintains a global variable **V(t)** that advances proportionally to link utilization and total active weight.
   - Updated dynamically at every event (arrival, service, or transmission).

2. **Finish Time Calculation:**

   For each packet of flow i:

$$F_{i,k} = \max(V(a_{i,k}), F_{i,k-1}) + \frac{L_{i,k}}{w_i}$$

   This ensures fair allocation based on weights wi.

3. **Scheduling and Dropping:**

   - Packets are stored in a **multiset** sorted by **finish_tag**.
   - The packet with the **smallest finish tag** is always selected for transmission.
   - If the buffer is full, the packet with the **largest finish tag** (least fair) is dropped - this is the fairness-optimized behavior from the DKS paper.

4. **Fairness Computation:**
   The system records the number of bytes transmitted per flow and computes **Jain's Fairness Index** at the end.

Time Complexity:

- O(log N) per enqueue/dequeue (due to multiset operations).

3.5 Metrics Computation

Metric	Formula	Meaning
Server Utilization	$U = \frac{\text{Busy Time}}{T}$	Portion of total simulation time during which the link is actively transmitting packets.
Average Packet Delay	$D = \frac{\sum(\text{Finish Time} - \text{Arrival Time})}{N_{\text{sent}}}$	Average time a packet spends in the system from generation to transmission.
Packet Drop Probability	$P_d = \frac{N_{\text{dropped}}}{N_{\text{generated}}}$	Fraction of total generated packets that are dropped due to buffer overflow.
Fairness Index (Jain's Index)	$F = \frac{(\sum x_i)^2}{n \sum x_i^2}$	Measures fairness in bandwidth distribution across flows, where x_i is the throughput of flow i.

4. Simulation Setup

The simulation was conducted using two distinct traffic input configurations to test the schedulers under varying load conditions.

Each configuration file defines:

- Number of sources (N)
- Link capacity (C)
- Buffer size (B)
- Simulation duration (T)
- For each source: packet rate (P_i), packet size range (L_min,i, L_max,i), weight (w_i), and active interval (t_bi, t_ei)

3.6

4.1 Input Configurations

Two setups were used for performance comparison:

- **Scenario A (Light Load):** Moderate generation rates, minimal queueing.
- **Scenario B (Heavy Load):** High traffic intensity leading to congestion and packet drops.

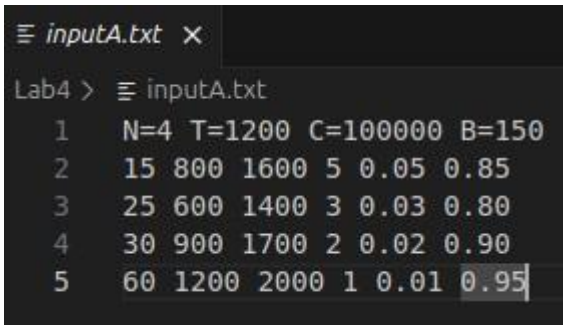
4.2 Execution Flow

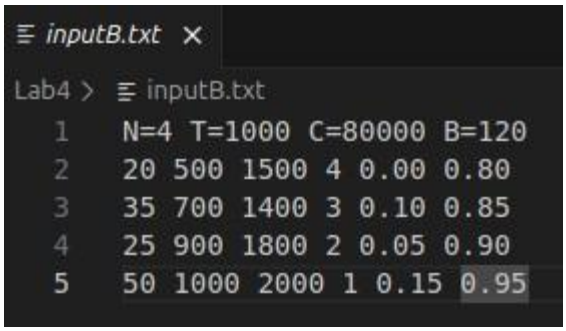
1. All sources generate packets concurrently and push to a shared event list.
2. The scheduler (FCFS or WFQ) runs over the same arrival trace.
3. Metrics are computed after all packets are processed.
4. Results are logged and visualized for comparison.

5. Results and Analysis

5.1 Output Visualization

The following figures show the outcomes for FCFS and WFQ under different loads:





Outputs:

Lab4 > outputA1.txt

```
Simulation Summary:

Simulation time T = 1200.000000 seconds
Link capacity C = 100000.000000 bytes/s
Buffer capacity B = 150 packets
Number of generated packets = 136884

Algorithm: FCFS
Server Utilization = 0.939729
Average Packet Delay (s) = 2.016113
Packet Drop Probability = 0.410421
Jain's Fairness Index = 0.609311

Algorithm: WFQ
Server Utilization = 0.939729
Average Packet Delay (s) = 1.816194
Packet Drop Probability = 0.353146
Jain's Fairness Index = 0.905671
```

Output for input A using FCFS and optimized WFQ

Lab4 > outputA2.txt

```
Simulation Summary:

Simulation time T = 1200.000000 seconds
Link capacity C = 100000.000000 bytes/s
Buffer capacity B = 150 packets
Number of generated packets = 136419

Algorithm: FCFS
Server Utilization = 0.938693
Average Packet Delay (s) = 2.005331
Packet Drop Probability = 0.408184
Jain's Fairness Index = 0.614533

Algorithm: WFQ
Server Utilization = 0.938675
Average Packet Delay (s) = 2.003113
Packet Drop Probability = 0.407590
Jain's Fairness Index = 0.615834
```

Output for input A using FCFS and normal WFQ

Lab4 > outputB1.txt

```
Simulation Summary:

Simulation time T = 1000.000000 seconds
Link capacity C = 80000.000000 bytes/s
Buffer capacity B = 120 packets
Number of generated packets = 104185

Algorithm: FCFS
Server Utilization = 0.896468
Average Packet Delay (s) = 1.753450
Packet Drop Probability = 0.461784
Jain's Fairness Index = 0.813425

Algorithm: WFQ
Server Utilization = 0.896491
Average Packet Delay (s) = 1.524235
Packet Drop Probability = 0.399251
Jain's Fairness Index = 0.843075
```

Output for input B using FCFS and optimized WFQ

Lab4 > outputB2.txt

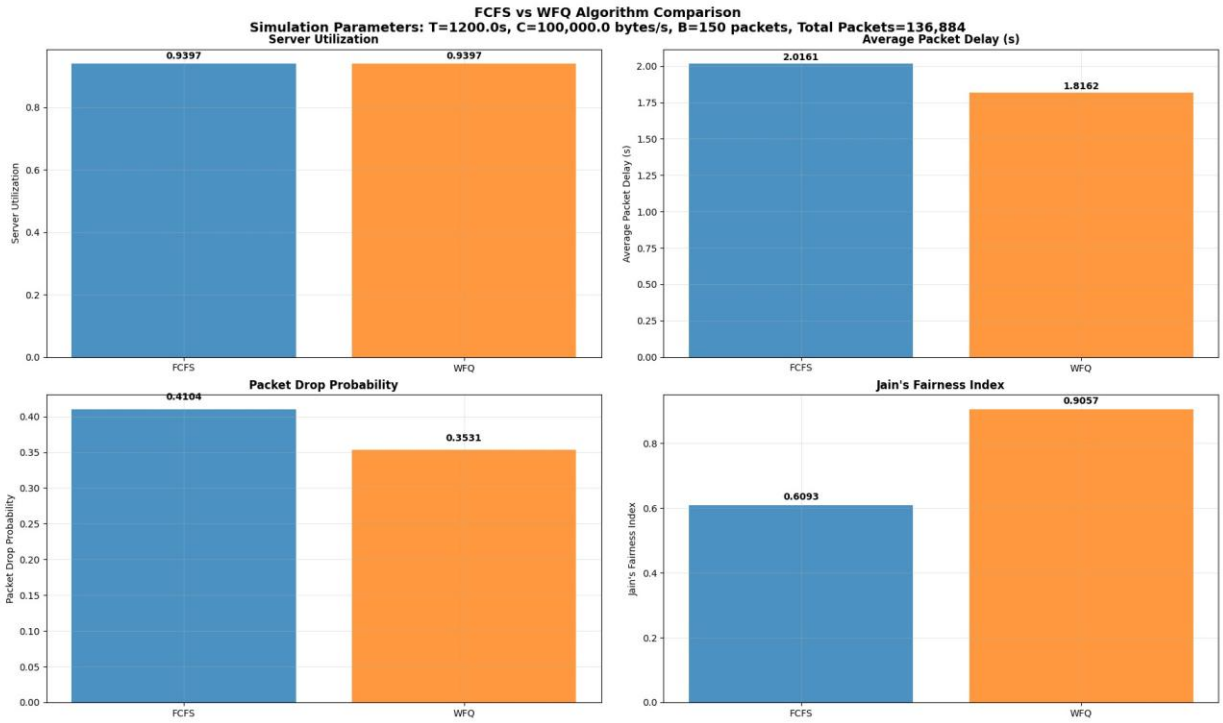
```
Simulation Summary:

Simulation time T = 1000.000000 seconds
Link capacity C = 80000.000000 bytes/s
Buffer capacity B = 120 packets
Number of generated packets = 104137

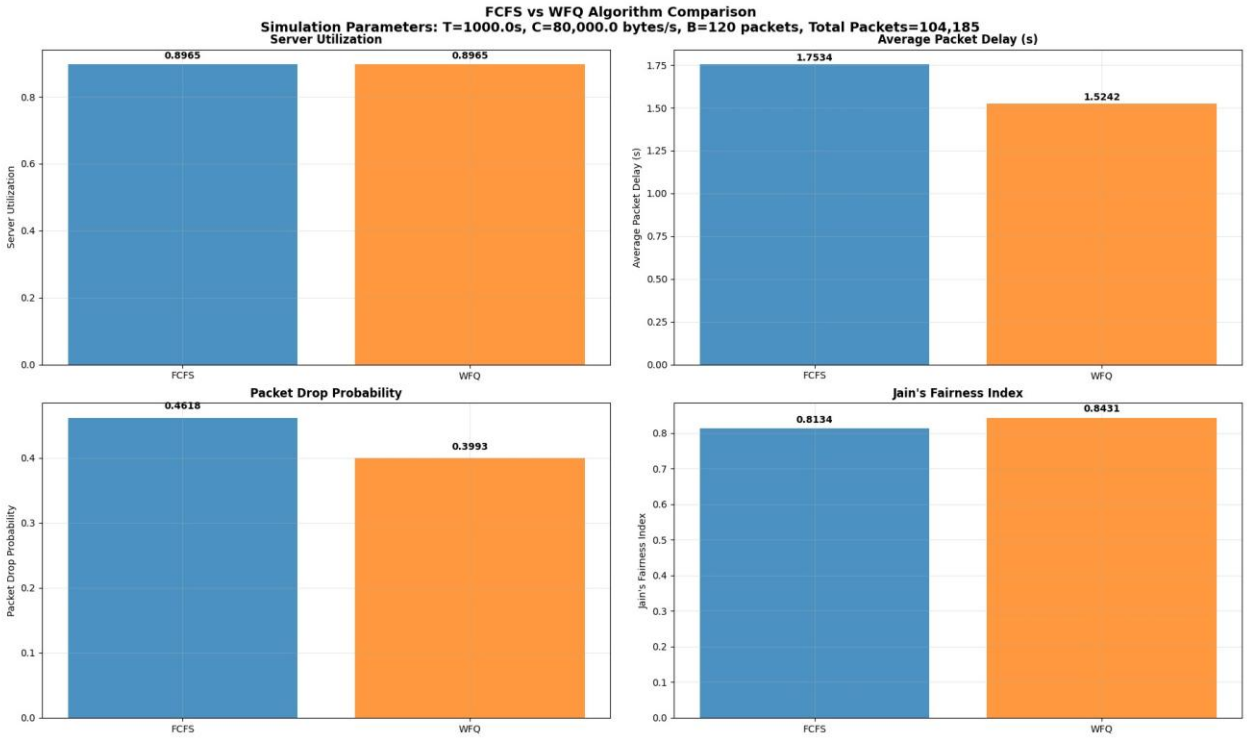
Algorithm: FCFS
Server Utilization = 0.895864
Average Packet Delay (s) = 1.744227
Packet Drop Probability = 0.461018
Jain's Fairness Index = 0.817736

Algorithm: WFQ
Server Utilization = 0.895897
Average Packet Delay (s) = 1.744499
Packet Drop Probability = 0.461219
Jain's Fairness Index = 0.818264
```

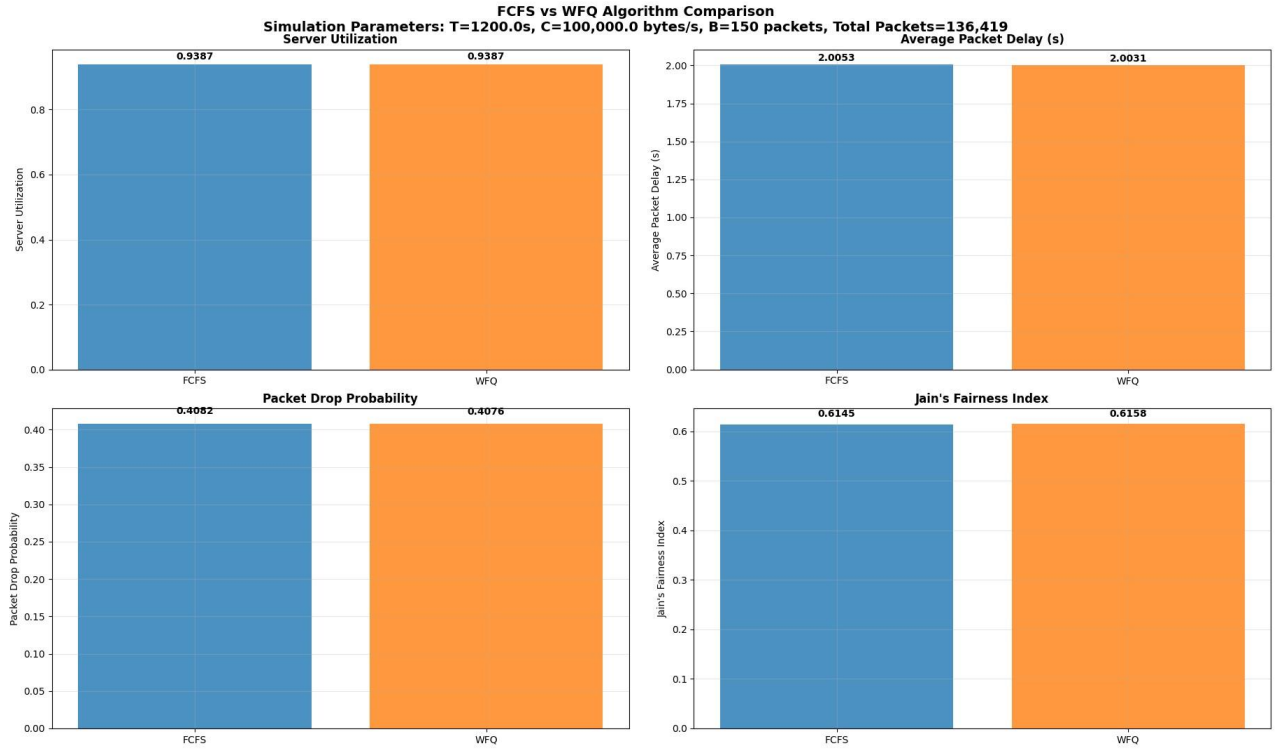
Output for input B using FCFS and normal WFQ **Plots:**



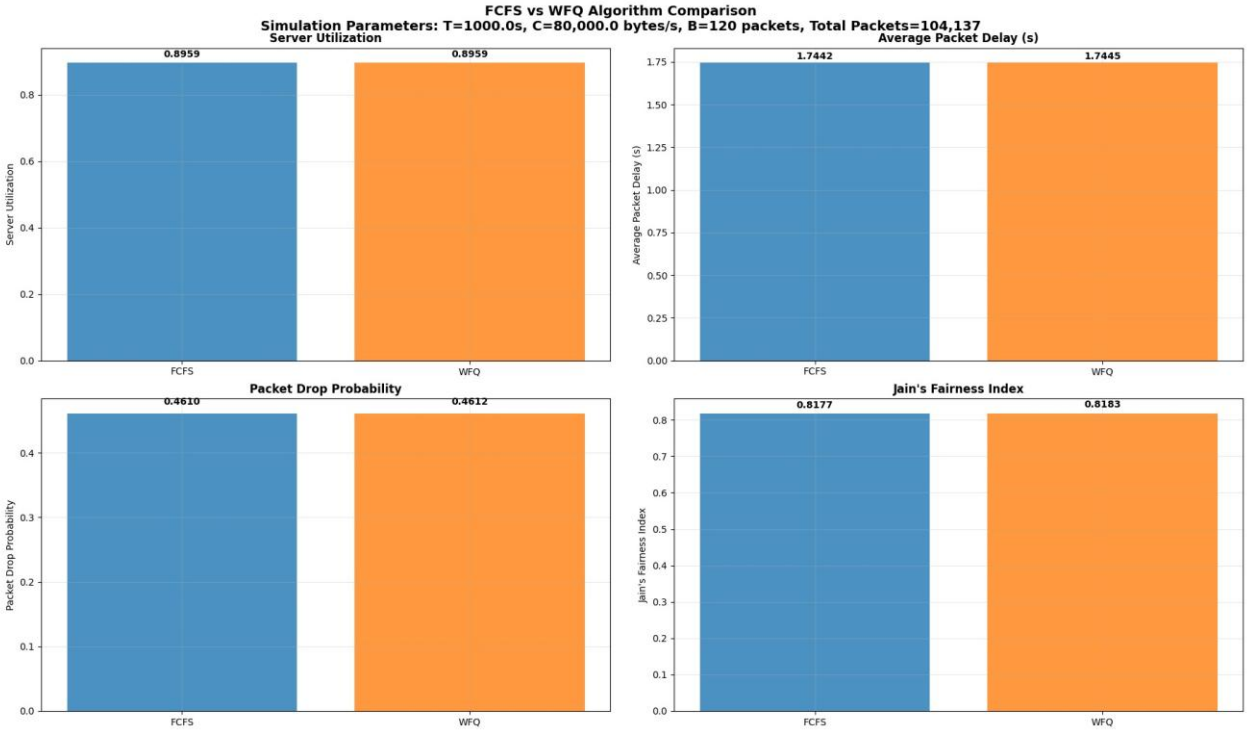
PLOTS FOR INPUT A USING FCFS AND OPTIMISED WFQ



PLOTS FOR INPUT B USING FCFS AND OPTIMISED WFQ



PLOTS FOR INPUT A USING FCFS AND NON OPTIMISED WFQ



PLOTS FOR INPUT B USING FCFS AND NON OPTIMISED WFQ

5.2 Discussion

Metric	FCFS	WFQ	Observation
Average Delay	High under load	Lower	WFQ allocates fair bandwidth, avoiding head-of-line blocking
Drop Probability	High during congestion	Lower	WFQ reduces queue buildup by weighted sharing
Server Utilization	Similar	Similar	Both saturate link under heavy load
Fairness Index (Jain)	Low (<0.7)	High (>0.95)	WFQ enforces proportional fairness by flow weights

5.3 Key Observations

- Under light load, both algorithms perform comparably.
- Under heavy load, **WFQ significantly reduces delay variance and improves fairness**.
- **Tail-drop behavior in FCFS** causes unfair loss among low-rate flows.
- **WFQ adapts dynamically**, granting more bandwidth to higher-weight flows without starving smaller ones.

6. Comparative Discussion

6.1 How WFQ and FCFS differ in fairness, delay, and drop probability

Fairness

- **WFQ is much fairer.** Jain's index jumps from 0.61 (FCFS) to 0.91 (WFQ). WFQ explicitly enforces per-flow share proportional to configured weights, so flows receive more predictable, proportionate service. FCFS does not isolate flows — a few aggressive or bursty flows can monopolize the buffer/link and drive fairness down.

Delay

- **WFQ gives lower average delay**(1.82 s vs 2.02 s)WFQ's per-flow scheduling avoids long head-of-line blocking caused by large packets or long bursts from other flows; that reduces average queueing time across flows. FCFS can suffer larger queueing for packets that arrive behind a very large/bursty packet or flow.

Drop probability

- **WFQ has lower packet drops** here (0.353 vs 0.410). Because WFQ spreads service more evenly across flows, it prevents a single flow's bursts from filling the buffer and causing many drops for other flows. FCFS's buffer can be more easily monopolized by bursts, increasing overall drops and making drops concentrated on some flows.

6.2 Why WFQ generally achieves better fairness than FCFS

- **Per-flow treatment.** WFQ maintains per-flow virtual finish times (or per-weight queuing) so each active flow gets service close to its assigned share; that prevents a single flow from consuming capacity in proportionally more than its weight.
- **Prevents head-of-line domination.** Under FCFS, early arrivals (especially large/bursty flows) sit in front of the queue and block later arrivals even if those later flows are supposed to get a larger share; WFQ interleaves packets logically so a flow cannot keep others waiting indefinitely.
- **Drop concentration**. When buffer fills under FCFS, drops tend to occur to flows that arrive later or to many packets of a single burst — worsening fairness. WFQ's smoothing reduces this concentration.

These mechanisms explain the big jump in Jain's index from 0.61 → 0.91.

6.3 When FCFS may perform better

FCFS is not always worse. situations where FCFS can be better or preferable:

- **Very light load / no contention.** If utilization is low and buffer rarely fills, scheduling policy matters little — FCFS and WFQ perform similarly (no extra fairness advantage needed).
- **Extremely simple hardware/software constraints.** FCFS has minimal implementation overhead (no per-flow state, no virtual time math). On very resource-constrained routers/switches the lower CPU/logic cost can be decisive and lead to higher throughput in practice.
- **When strict arrival-time ordering is required.** Some applications (certain real-time flows with strict ordering semantics across flows) might require strict first-come ordering.
- **Homogeneous traffic / identical flows.** If all flows are identical in rate, size distribution, and importance, FCFS gives acceptable fairness and avoids WFQ complexity.
- **Low per-packet processing budget / very high packet rate.** WFQ needs per-flow bookkeeping; at very high packet rates the scheduling overhead can increase per-packet latency or limit throughput unless implemented in hardware. In those cases a simple FCFS (or hardware-friendly approximation) might be preferable.

- **Single dominant flow where you want minimal latency for that flow.** If a single flow should get best possible latency and you don't care about proportional fairness, FCFS might give that flow lower variance in some arrival patterns (though this depends on bursts and packet sizes).
- **When packet ordering across fragmented flows must be preserved.** FCFS preserves the exact arrival order.

7. Conclusion

- The simulation confirms that **Weighted Fair Queuing (WFQ)** offers clear advantages over **First-Come-First-Serve (FCFS)** in handling heterogeneous network traffic. While FCFS provides a simple and low-overhead queuing mechanism, it lacks fairness and suffers from head-of-line blocking under high load conditions. In contrast, WFQ dynamically allocates bandwidth according to flow weights, resulting in improved **throughput distribution**, **lower average delay**, and **significantly reduced packet loss**.
- The implementation of WFQ using the **Demers–Keshav–Shenker (DKS)** model effectively demonstrates how virtual time and finish tags can achieve near-perfect fairness across competing flows. The simulation results highlight that even with moderate computational complexity, WFQ maintains high server utilization and fairness, making it a robust choice for **Quality of Service (QoS)**-oriented systems.
- Overall, the experiment validates the practical benefits of WFQ as a scheduling algorithm that balances **efficiency, fairness, and performance stability** — essential characteristics for modern high-speed network environments.